Universidad Nacional de Loja

Facultad de la Energía, las Industrias
y los Recursos Naturales No Renovables

Carrera de Computación




# Síntesis: Latencia de Almacenamiento y su Impacto en el Software

**Asignatura:** Estructura de Datos

**Docente:** Ing. Cristián Narváez

*Autor:*

Axel Joao Jimenez Palta

Octubre de 2026

# 1. Jerarquía del Almacenamiento y Brecha en la Velocidad

La posición de los datos dentro de jerarquía de memoria del computador es quién condiciona el tiempo de respuesta de cualquier aplicación. Las latencias en la jerarquía están distribuidas de esta manera:

| **Componente de Almacenamiento** | **Tiempo de Acceso (Latencia)** | **Comparación Directa contra la RAM** |
|---|---|---|
| Registros y Caché L1 (CPU) | 1 nanosegundo (ns) | 100 veces más rápido que la RAM |
| Caché L2 Y L3 (CPU) | 4 a 15 nanosegundos (ns) | 7 a 25 veces más rápido que la RAM |
| Memoria Principal (RAM) | 100 nanosegundos (ns) | Punto de Referencia |
| SSD NVMe | 100 microsegundos ($\mu$s) | 1,000 veces más lento que la RAM |
| SSD SATA | 100 a 500 microsegundos ($\mu$s) | 1,000 a 5,000 veces más lento que la RAM |
| Disco Duro Mecánico (HDD) | 10 milisegundos (ms) | 100,000 veces más lento que la RAM |

Esto quiere decir que la CPU procesa millones de operaciones por segundo mientras los datos están en la RAM. Entonces si el software está buscando un dato en el almacenamiento secundario, la velocidad de procesamiento se ve reducida de manera inmediata por un obstáculo físico de órdenes de magnitud.

# 2. La Pila I/O del Sistema Operativo y los Mecanismos de Suspensión de Hilos

Cuando una aplicación realiza alguna petición síncrona como una llamada de lectura `read()` pausa el hilo y cambia de contexto por que como el software no puede continuar sin el dato solicitado, la CPU pone en estado de reposo al hilo de la aplicación, luego el sistema operativo realiza un cambio de contexto para asignar ese núcleo de CPU a otra tarea mientras el disco responde. Una vez el controlador NVMe completa la lectura física en el medio interno, envía una interrupción de hardware que notifica al Kernel.

El Kernel ejecuta la rutina de servicio de interrupción ISR que levanta el hilo, pero no se ejecuta de inmediato, primero debe entrar a la cola de ejecución del planificador de procesos y esperar que se le reasigne un tiempo de CPU, lo que provoca un retraso crítico en la ruta de espera.

Por otro lado en sistemas de archivos como ext4, realizar una llama `fsync()` para garantizar la durabilidad de un archivo genera 4 operaciones físicas de disco y provoca hasta 6 cambios de contexto, lo que multiplica el tiempo de espera del software.

# 3. El Colapso por Congestión de Colas e Interferencia

Al existir múltiples peticiones o usuarios interactuando simultáneamente con la aplicación ocurre la "Saturación de colas", que si la tasa de peticiones entrantes supera las operaciones por segundo (IOPS) del disco, las solicitudes se acumulan en las colas del kernel (blk-mq) y del controlador del dispositivo. Una profundidad de cola elevada viene a destruir la baja latencia del dispositivo SSD. Si en el mismo servidor coinciden procesos en segundo plano y aplicaciones interactivas en línea las ráfagas E/S del proceso pesado llegan a congestionar el disco. Esto causa que, aunque el tiempo medio de respuesta parezca ser estable, el Percentil 99 y 99.9 de los usuarios sufra retrasos exponenciales de decenas o cientos de milisegundos.

# 4. Ejemplos de Consecuencias Prácticas en el Software

- **Fallos de Páginas en Base de Datos:** En motores de base de datos (MySQL, PostgreSQL, etc.) si el volumen de datos excede la memoria RAM, cada consulta que no encuentre su página en el buffer pool genera un fallo de página que obliga a leer el bloque desde el SSD/HDD, lo que

provoca que se desplome la capacidad de procesamiento de la base de datos de millones a solo cientos o miles transacciones por segundo.

- **Servidores web:** En servidores web, los hilos de trabajo se quedan bloqueados en estado de "I/O wait" esperando al almacenamiento. En el momento que se agoten los hilos disponibles en el servidor, este deja de aceptar nuevas conexiones, lo que genera errores de tiempo de espera agotado para los usuarios.
- **Bloqueos en Cascada Distribuida:** Si un dispositivo de almacenamiento experimenta una degradación física leve y reduce su velocidad por órdenes de magnitud sin fallar del todo, retiene las peticiones de manera indefinida, lo que acumula colas de mensajes y provoca errores fuera de memoria y caídas en cascada por todo un clúster distribuido.

# 5. Conclusiones

- Concluyo que la latencia de almacenamiento es el cuello de botella primario del hardware, ya que la brecha de hasta cuatro órdenes de magnitud entre la memoria RAM y los medios secundarios limita a un nivel significativo el rendimiento real del software.
- Se evidencia que los modelos de I/O síncronos disminuyen drásticamente la respuesta del sistema, por el motivo de que las operaciones de almacenamiento bloqueantes fuerzan a la CPU a poner los hilos en reposo lo que provoca cambios de contexto y retrasos en la planificación del Kernel.
- Por último, se concluye que la latencia de almacenamiento ha dejado de ser un asunto exclusivo de hardware para convertirse en un principio fundamental de la arquitectura de Software. La velocidad y estabilidad de un sistema no se consigue esperando que los discos sean más rápidos, sino construyendo software consciente de la jerarquía de memoria que este diseñado para minimizar la fricción de I/O, evitar los bloqueos en la CPU y optimizar cada interacción con el hardware.

# 6. Referencias


[1] J. Dean y P. Norvig, "Latency numbers every programmer should know," Google Research / Stanford Univ., 2012. [En línea]. Disponible en: `https://introcs-python.cs.luc.edu/hardware/overview.html`

[2] G. Haas y V. Leis, "What modern NVMe storage can do, and how to exploit it: High-performance I/O for high-performance storage engines," Proc. VLDB Endow., vol. 16, no. 9, pp. 2090–2102, mayo 2023. [En línea]. Disponible en: `https://www.vldb.org/pvldb/vol16/p2090-haas.pdf`

[3] Q. Zhang et al., "FLASHSHARE: Punching through server storage stack from kernel to firmware for ultra-low latency SSDs," en Proc. 13th USENIX Symp. Oper. Syst. Des. Impl. (OSDI 18), Carlsbad, CA, EE. UU., 2018, pp. 477–492. [En línea]. Disponible en: `https://www.usenix.org/system/files/osdi18-zhang.pdf`

[4] L. Hu, H. Liu et al., "Towards low-latency I/O services for mixed workloads using ultra-low latency SSDs," en Proc. 36th ACM Int. Conf. Supercomput. (ICS '22), junio 2022, pp. 1–12. [En línea]. Disponible en: `https://people.ucsc.edu/~lhu82/Biobibnet/22ICS_FastResponse.pdf`

[5] J. Arulraj, A. Pavlo, y W. C. Tan, "Write-behind logging," Proc. VLDB Endow., vol. 9, no. 4, pp. 337–348, diciembre 2015. [En línea]. Disponible en: `https://db.cs.cmu.edu/papers/2016/p337-arulraj.pdf`

[6] H. S. Gunawi et al., "Fail-slow at scale: Evidence of hardware performance faults in large production systems," en Proc. 16th USENIX Conf. File Storage Technol. (FAST 18), Oakland, CA, EE. UU., 2018, pp. 223–239. [En línea]. Disponible en: `https://www.usenix.org/system/files/conference/fast18/fast18-gunawi.pdf`


# 7. Uso y Justificación de IA

Para este trabajo he utilizado la IA "Gemini Notebook", para cargar mis propias fuentes en la plataforma para que me ayudara a extraer, procesar y sintetizar toda la información clave directamente de los documentos consultados. También la usé para pedir sugerencias sobre cómo estructurar mejor los temas, apoyándome en ella también para mejorar la redacción final.

El prompt utilizado fue el siguiente:
*"Para mi presente trabajo, actúa como un guía experto en Estructura de Datos, sintetiza la información proporcionada por mis fuentes para poder realizar un ensayo sobre cómo la latencia de almacenamiento influye en la velocidad de respuesta del software, dame consejos y sugerencias de como estructurar mejor los temas, y conforme suba mis borradores, ayúdame en la redacción."*